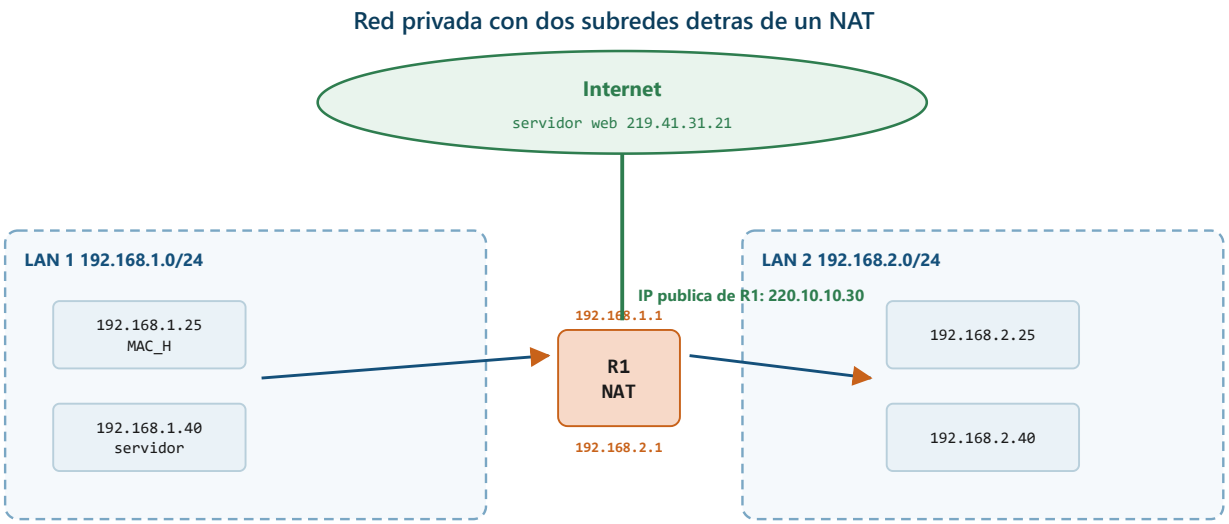
Clase 31 — NAT, PAT y sus hacks



Bloque: D (Capa de Red) · **Semana:** 11 · **Duración estimada:** 120 min **Correlativa:** Clase 30 (tablas de enrutamiento, next hop y ARP) · **Sigue con:** Clase 32 (enrutamiento por vector de distancias) **Material de referencia:** Tanenbaum cap. 5 · Exámenes `sirc1102`, `sirc1103`, `sirc0812` · RFC 8200, RFC 7915

> 🔍 = RFC **no verificado** en `ACTUALIZACIONES.md`: no lo cites sin confirmarlo.

🎯 Al terminar esta clase vas a poder

Tabla NAT con traduccion de puertos (PAT)

IP privada : puerto	IP publica : puerto	estado
192.168.1.25 : 51000	220.10.10.30 : 40001	activa
192.168.1.25 : 51002	220.10.10.30 : 40002	activa
192.168.1.40 : 33333	220.10.10.30 : 40003	activa
192.168.1.77 : 8080	220.10.10.30 : 40004	timeout 300 s

La clave de la tabla es (IP privada, puerto privado). El puerto es lo que permite multiplexar many-to-one: 3 sesiones, 1 sola IP publica.

Al volver un paquete se busca (IP publica, puerto publico) y se traducen la IP destino y el puerto destino.

Ese es el motivo por el que NAT rompe el aislamiento de capas: el router tiene que entender TCP/UDP para funcionar.

1. **Explicar** por qué existe la NAT: escasez de IPv4 y direcciones privadas que no se rutean.
2. **Distinguir** traducción de **dirección** de traducción de **puerto** (PAT / NAPT), y armar una tabla NAT con varias sesiones simultáneas.
3. **Describir las tres fases**: salida, entrada y *timeout*.
4. **Describir campo por campo** qué cambia un router NAT en cada sentido, y resolver el `2011-03 Problema 2c`.
5. **Justificar** por qué la NAT **rompe el aislamiento de capas**, que es el `2011-02 P4b`.
6. **Explicar las tres soluciones** para que un cliente con IP pública alcance a un servidor privado, que es el `2011-02 P4c`.
7. **Explicar** el *hairpinning* y el **CGNAT** con sus tres problemas.
8. **Resolver entero** el `2008-12 Problema 1` (30 pts): NAT para balanceo de carga.

> **Prerequisito del examen:** `2011-03 Problema 2 c` (NAT + tabla), `2011-02 P4` (10 pts), `2008-12 Problema 1` (30 pts), `2010-03 P1` (subredes privadas y NAT).

PARTE 1 — TEORÍA

1.1 Qué es la NAT y por qué apareció

NAT (*Network Address Translation*) es una función de un **router** que **reescribe las direcciones IP** de los paquetes que lo atraviesan. Las dos razones por las que existe, y hay que decir las dos:

```
1. ESCASEZ DE IPv4.   Hay 2³² ≈ 4 300 millones de direcciones y el mundo
                      las consumió: el RIPE NCC hizo su última asignación
                      /22 el 25-nov-2019. APNIC se agotó en abr-2011, ARIN
                      en sept-2015, LACNIC en 2020 y AfriNIC en abr-2017.

2. DIRECCIONES PRIVADAS QUE NO SE RUTEAN.  10.0.0.0/8, 172.16.0.0/12 y
                     192.168.0.0/16 son de uso interno y NO se propagan por
                     Internet, así que una empresa puede reutilizar el mismo
                     direccionamiento en todas sus sedes.
```

El punto clave: la NAT **no es seguridad**, es **escasez**. Se hizo "hacia atrás" (era un truco de transiciones) y terminó siendo la razón por la que IPv4 sobrevivió quince años más.

⚠️ Actualización a 2026

Sigue igual: los tres bloques privados, la idea de la traducción y los puertos privados.

Cambió: - **El agotamiento de IPv4 es un hecho consumado.** RIPE hizo su última asignación `/22` el **25-nov-2019** y hoy mantiene una *waiting list* para un único `/24` recuperado. - **IPv6 dejó de ser "el futuro".** El **28 de marzo de 2026** el acceso a Google por IPv6 **superó el 50 % por primera vez (50,10 %)**. Pero es desigual: Francia e India por encima del 70 %, China por debajo del 5 %. **La cifra depende de qué se mida**: Cloudflare (tráfico por bytes) rondaba el 40 %. - **La meta de IPv6 es eliminar la NAT**, porque rompe la **extensibilidad extremo a extremo**, no es *stateful-friendly* con IPsec, y obliga a los hosts IPv6-only a usar 464XLAT para alcanzar contenido IPv4. - **NAT64 (`RFC 6146`) y DNS64 (`RFC 6147`)** maduraron con el prefijo `64:ff9b::/96` (`RFC 6052`) y `RFC 8880`. **SIIT (`RFC 6145`) quedó obsoleto por `RFC 7915`.** **464XLAT sigue en borrador**: la transición no cerró.

1.2 Traducción de dirección contra traducción de puerto

```
TRADUCCIÓN DE DIRECCIÓN SOLA
   H1 .10 ──► R1 .200 ──► S:80          ⇐ UNA sola sesión: todas salen
                                           iguales desde afuera. Y un
                                           navegador abre 6 al mismo servidor.

TRADUCCIÓN DE DIRECCIÓN + PUERTO (PAT / NAPT)
  ┌──────┐                                          ┌──────┐
  │ H1   │ .10:40000 ──► R1 .200:50001 ──►  │  S   │ :80
  │ H1   │ .10:40001 ──► R1 .200:50002 ──►  │      │
  │ H2   │ .11:40000 ──► R1 .200:50003 ──►  │      │
  └──────┘                                          └──────┘

⇐ Cada sesión tiene un PAR (IP pública, puerto público) ÚNICO, así que R1
   soporta miles simultáneas (hasta ~64 000 por IP pública).
```

El nombre correcto: `PAT` (*Port Address Translation*) o `NAPT`. **En los exámenes de FING, "NAT" significa PAT**: se cambia IP **y** puerto.

La tabla NAT, con 3 sesiones simultáneas

```
┌──────────────────────────────────────────────────────────────────────┐
│  (IP origen, puerto origen)   →  (IP pública, puerto público)        │
│  ──────────────────────────       ──────────────────────────────     │
│  (192.168.1.25, 40000)        →  (220.10.10.30, 50001)               │
│  (192.168.1.25, 40001)        →  (220.10.10.30, 50002)               │
│  (192.168.1.26, 40000)        →  (220.10.10.30, 50003)               │
│                                                                      │
│  La clave es el PAR interno; el valor es el PAR externo.             │
└──────────────────────────────────────────────────────────────────────┘
```

Las tramas de los dos sentidos, para la sesión de `192.168.1.25:40000`:

```
⇐ SALIENTE (H → servidor)          ⇐ ENTRANTE (servidor → H)

IP   192.168.1.25   TCP src 40000   IP   220.10.10.30   TCP src    80
IP   219.41.31.21   TCP dst    80   IP   219.41.31.21   TCP dst 50001
          ──NAT──►                                ◄──NAT──
Eth  MAC_H1 → MAC_R1                  IP   192.168.1.25   TCP src    80
                                       Eth  MAC_R1 → MAC_H1  TCP dst 40000
```

⚠️ **El campo que el router mira para decidir el sentido es el puerto de destino.** Por eso hace falta traducir puertos: la IP de destino sola no distingue tres sesiones al mismo servidor.

1.3 Las tres fases de la NAT

```
1. SALIDA     Llega paquete de la LAN. ¿(IP priv, port priv) está en la
               tabla?  NO → SE CREA entrada con puerto público nuevo.
2. ENTRADA    Llega paquete de Internet. ¿(IP pub, port pub) está en la
               tabla?  NO → SE DESCARTA.  SÍ → se traduce destino y puerto.
3. TIMEOUT    Pasan TIMEOUT s sin tráfico en esa entrada → se BORRA y el
               puerto público vuelve al pool.
```

Por qué hace falta el timeout: sin él la tabla crecería sin límite. **Pero** es un compromiso: corto, se pierde la correspondencia de una conexión larga; largo, la tabla se llena. Eso se llama **UDP hole punching** y es lo que rompe al P2P.

1.4 Por qué la NAT rompe el aislamiento de capas

Este es el `2011-02 P4b` (10 puntos) y es **el mejor punto conceptual de la clase**.

```
El AISLAMIENTO DE CAPAS consiste en que cada capa use a la de abajo como un
servicio opaco. En particular, la capa 3 decide el reenvío mirando SOLO la
dirección IP destino. ⇐ Un router NO necesita entender TCP para reenviar un
paquete TCP.

──────────────────────────────────────────────────────────────────────

CON NAT, un paquete que ENTRA al router tiene:

     IP destino = 220.10.10.30    ← es la IP DEL PROPIO ROUTER, y es ÚNICA
                                     para toda la LAN

⇐ La IP destino NO dice a qué máquina interna va. La única información que lo
   dice es el PUERTO DE DESTINO:

     puerto destino = 50001 → buscar en la tabla → 192.168.1.25:40000

⇐ Para decidir eso el router tiene que LEER EL PUERTO TCP/UDP, campo de la
   CABECERA TCP: es información de CAPA 4. Un dispositivo de capa 3
   dependiendo de un protocolo de capa 4
   ⇐ ESO ES ROMPER EL AISLAMIENTO DE CAPAS.
```

Las tres consecuencias concretas, que son lo que suma puntos:

```
1. ROMPE LA EXTENSIBILIDAD.  Cada capa debe poder cambiar sin romper las de
   arriba; con NAT no alcanza con "dejar pasar" lo que no se entiende: hay
   que INTERPRETARLO.
2. ROMPE LA TRANSPARENCIA.  El cliente ve su socket en 192.168.1.25:40000 y
   el servidor ve 220.10.10.30:50001. Ninguna es la dirección "verdadera":
   eso rompe IPsec en modo ESP (que cifra las cabeceras IP).
3. ROMPE EL P2P.  Como la traducción solo se crea al INICIAR una conexión
   saliente, un host detrás de la NAT no puede iniciar nada hacia afuera
   por su cuenta (ver §1.5).
```

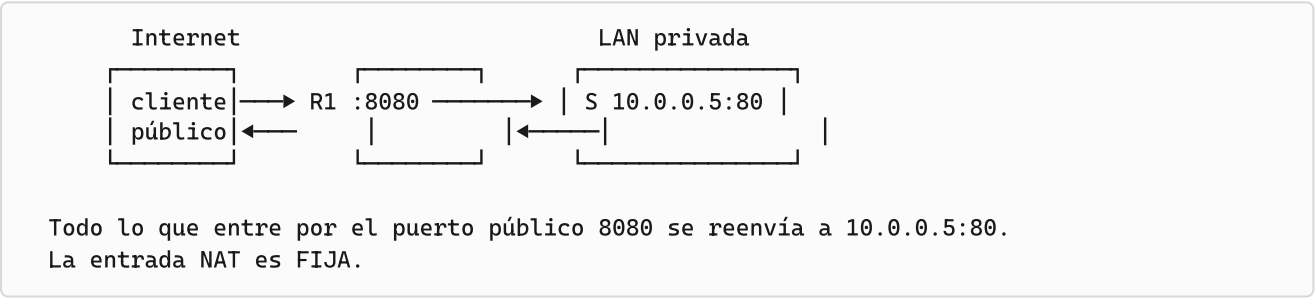
La frase de cierre, que es lo que el corrector quiere leer:

> La NAT rompe el aislamiento de capas porque un router que solo debería mirar la dirección IP destino se ve obligado a **mirar el puerto TCP/UDP** para saber a qué host interno corresponde el paquete. Eso convierte a un dispositivo de capa 3 en uno que depende de un protocolo de capa 4, y por lo tanto **cualquier protocolo nuevo de capa superior que no sea TCP o UDP deja de funcionar detrás de una NAT** sin que el router lo entienda.
>
> ⚠️ **Errores conceptuales típicos:** (i) decir que rompe el aislamiento porque "el router mira la IP privada" — falso, la IP es de capa 3 y es lo que siempre hace; lo que rompe es mirar **el puerto**; (ii) decir que rompe porque "el servidor ve otra IP" — eso es el síntoma, no la causa; (iii) olvidar el caso **ESP**.

1.5 Las tres soluciones para que un cliente con IP pública alcance a un servidor privado

Este es el `2011-02 P4c`. Hay que presentarlas **como están**, pero nombrándolas bien.

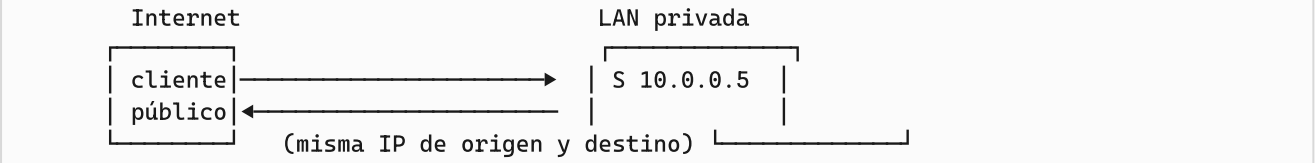
La solución 1 — *Port forwarding* (NAT estática, o DNAT)




```
Todo lo que entre por el puerto público 8080 se reenvía a 10.0.0.5:80.
La entrada NAT es FIJA.
```

Requisitos: el servidor privado necesita **IP privada estable** (un DHCP *reservation* alcanza) y hay que **saber de antemano** puerto y protocolo. **No sirve para conexiones salientes de la LAN**: solo traduce en un sentido.

La solución 2 — NAT sin traducción (*full-cone NAT* o *endpoint-independent mapping*)




```
⇐ El servidor tiene una dirección que el cliente PÚBLICO puede usar
   directamente, o hay un mapeo 1:1 declarado de antemano.
```

Cuándo se usa: cuando el servidor tiene dirección pública propia pero su **gateway por defecto** es un router que hace NAT (una máquina en la nube con IP elástica, por ejemplo). **Limitación:** alguien tiene que **configurar el mapeo de antemano**; si no, el paquete nunca vuelve (ver §1.6).

La solución 3 — El servidor inicia: invertir quién origina la conexión

```
EL SERVIDOR PRIVADO ABRE UNA CONEXIÓN SALIENTE; EL ROUTER GUARDA LA
ENTRADA Y EL CLIENTE PÚBLICO CONTESTA A ESA CONEXIÓN.

paso 1 (SALIENTE)  10.0.0.5:40000 → R1 220.10.10.30:50007 → S:80
                   se CREA la entrada NAT; el servidor externo abre la
                   conexión de retorno.
paso 2 (ENTRANTE)  CLIENTE → R1 220.10.10.30:50007 → 10.0.0.5:40000
                   se busca (IP pub, port pub) en la tabla y se traduce.
```

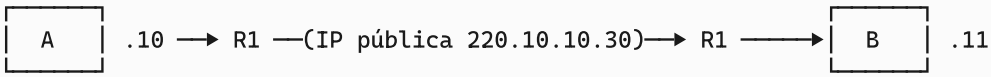
Por qué es la más usada: la NAT solo traduce lo que **sale**. Si el servidor privado manda primero un paquete (un keep-alive, un `REGISTER` de SIP, un *binding request* de STUN), la entrada queda creada y **cualquier paquete que entre desde Internet con ese par** se reenvía adentro. **Es la base de SIP, ICE (WebRTC), STUN y TURN**.

	1. *Port forwarding*	2. NAT sin traducción	3. El servidor inicia
Quién configura	el administrador	el administrador	**nadie**
Tabla NAT	entrada fija	no hay traducción	entrada **dinámica**
IP del servidor	privada	mapeo 1:1 declarado	privada
Sin tocar nada	no	no	**sí**
Uso típico	servidores web, cámaras	IP elástica	**VoIP, WebRTC, P2P**

Regla mental: si **alguien tiene que configurar algo**, es la solución 1 o la 2; si **no hay que configurar nada**, es la 3.

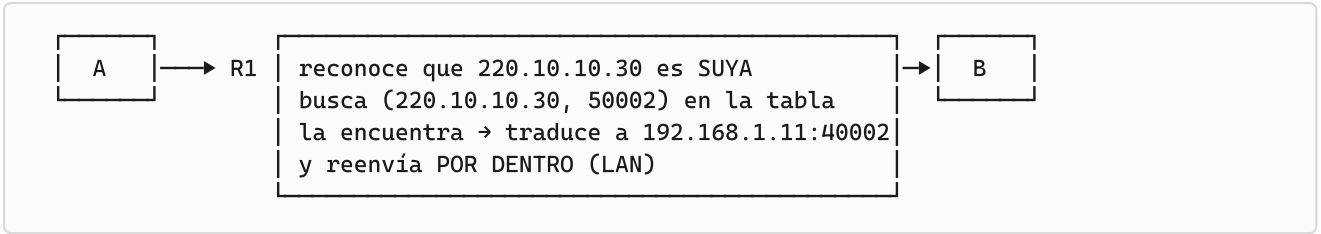
1.6 *NAT hairpinning* (el bucle hacia atrás)

El problema. Un cliente de la LAN quiere alcanzar a **otro cliente de la misma LAN** a través de la **IP pública** del router:



⇐ A manda a 220.10.10.30:50002. El paquete **llega físicamente al router**, porque esa IP pública es de R1. La entrada NAT de B existe (se creó cuando B salió), así que **debería** funcionar. ⚠️ PERO muchos routers NO lo hacen: la entrada se creó para tráfico hacia afuera, y se descarta por seguridad todo paquete que llega por la INTERFAZ EXTERNA con destino una IP pública PROPIA.

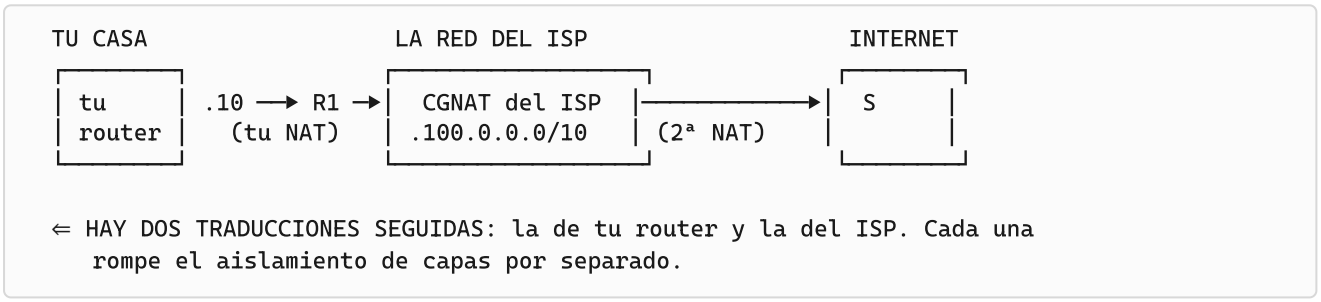
La solución: el *hairpin* (o *NAT loopback*). El router reconoce que **el destino es él mismo** y, en vez de rechazar, **traduce igualmente** y reenvía por la interfaz **interna**:



Sin hairpin se rompen dos casos: un servidor interno que publica un servicio en su IP pública y es accedido por otro equipo de la misma LAN, y **el P2P entre dos equipos de la misma LAN**, que directamente no se ven.

1.7 CGNAT: la NAT del operador

CGNAT (*Carrier-Grade NAT*) es la misma técnica, pero **aplicada por el operador** a toda la base de clientes:



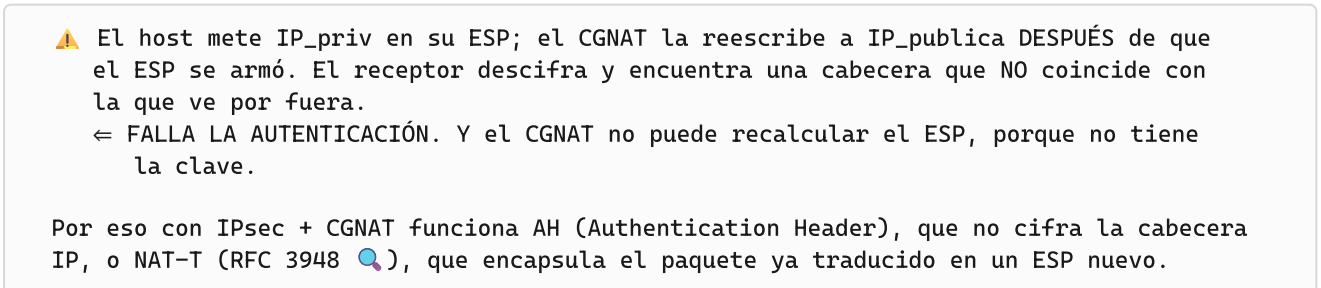
Por qué se volvió ubicuo: por el mismo motivo que la NAT, y con más fuerza. Cuando el RIPE hizo su última asignación en 2019, los operadores dejaron de poder entregar una IP pública a cada casa y pasaron a entregar un prefijo de la **red privada del operador**, con CGNAT.

Los tres problemas del CGNAT

Problema 1 — Rompe la conectividad entrante. La tabla del CGNAT **solo se llena cuando el cliente sale**, y sin *port forwarding* **nadie de Internet puede iniciar una conexión** hacia tu casa. Impacto: **VoIP, juegos, SSH, servidores domésticos y cámaras IP**.

Problema 2 — Complica la señalización. SIP/IMS mandan la dirección **pública** dentro de los mensajes y usan cabeceras `Contact`, `Via` y `Record-Route` para saber por dónde volver. Con CGNAT esa dirección es **la del CGNAT**, y los elementos de ruta apuntan a una dirección que **no existe** en la red del operador: hay que agregar un **outbound proxy**.

Problema 3 — Rompe IPsec en ESP. En modo **ESP** el cifrado cubre la cabecera IP completa:



> ⚠️ **RFCs no verificados:** la especificación clásica de NAT tradicional (`RFC 3022`), el *shared address space* del CGNAT (`RFC 6598`), los requisitos del CGN (`RFC 6888`) y el NAT-T (`RFC 3948`) **NO están en** `ACTUALIZACIONES.md` : **verificar** en `rfc-editor.org` antes de citarlos. Lo verificado de esta clase: `RFC 8200` , `RFC 6146` , `RFC 6147` , `RFC 6052` , `RFC 8880` , `RFC 7915` , `RFC 6333` , `RFC 9000` .
>
> **El dato de 2026 que reencuadra todo:** el **NAT rebinding** (el operador cambia el puerto de la tabla a mitad de una conexión) **rompe TCP**, y por eso los clientes reconectan (`RFC 8443` 🔍). **Con QUIC no rompe nada**, porque el **Connection ID** (`RFC 9000` §5.1) identifica la conexión con independencia de las direcciones. **Ése es el cambio de paradigma:** la identidad de la conexión deja de ser la IP.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — Armar la tabla NAT con 4 sesiones simultáneas

Datos. Router `R1` con IP pública `220.10.10.30`. LAN `192.168.1.0/24`. Cuatro conexiones salientes simultáneas, todas hacia `219.41.31.21:80`.

#	Host interno	Puerto origen	Puerto público asignado
1	`192.168.1.25`	1024	50001
2	`192.168.1.25`	1025	50002
3	`192.168.1.26`	1024	50003
4	`192.168.1.30`	51000	50004

La tabla NAT completa:

```
┌────┬──────────────────────────┬──────────────────────────────────┬─────────┐
│ #  │ (IP origen int, port int) │ → (IP pública, port público)     │ estado  │
├────┼──────────────────────────┼──────────────────────────────────┼─────────┤
│  1 │ (192.168.1.25, 1024)     │ → (220.10.10.30, 50001)          │ ACTIVA  │
│  2 │ (192.168.1.25, 1025)     │ → (220.10.10.30, 50002)          │ ACTIVA  │
│  3 │ (192.168.1.26, 1024)     │ → (220.10.10.30, 50003)          │ ACTIVA  │
│  4 │ (192.168.1.30, 51000)    │ → (220.10.10.30, 50004)          │ ACTIVA  │
└────┴──────────────────────────┴──────────────────────────────────┴─────────┘
```

Las tres observaciones que hacen los puntos: (1) el mismo host `192.168.1.25` tiene **DOS entradas** porque el puerto de origen es distinto (1024 y 1025). (2) El mismo puerto (1024) lo usan **dos hosts distintos** (`.25` y `.26`) sin conflicto, porque la clave es el **PAR**. (3) Todos los puertos públicos son distintos aunque la IP pública sea una sola.

Las tramas de los dos sentidos para la sesión 3 (la que repite puerto interno):

```
⇐ H → S (salida)                    ⇐ S → H (entrada)

IP  192.168.1.26   TCP src 1024      IP  220.10.10.30   TCP src    80
IP  219.41.31.21   TCP dst   80      IP  219.41.31.21   TCP dst 50003
Eth MAC_H26 → MAC_R1                          NAT⟶ IP 192.168.1.26  TCP src 80
                 ⟶NAT⟶              Eth  MAC_R1 → MAC_H26   TCP dst 1024
```

2.2 Ejercicio 2 — ¿Por qué no funciona el P2P detrás de un CGNAT?

```
Paso 1: A (LAN hogareña) manda su descubrimiento a B. Sale por su router
        → se crea la entrada NAT de A.

Paso 2: B (detrás del CGNAT) manda su descubrimiento a A.
        ⇐ FALLA AQUÍ. El CGNAT residential no ofrece port forwarding: la
           entrada de B existe solo para tráfico SALIENTE, así que nadie
           de Internet puede iniciar nada hacia B.

Paso 3: aunque el paquete de B llegara, el de A no llegaría a B, porque el
        CGNAT no tiene entrada para (IP pública de A, puerto destino).

⇐ CONCLUSIÓN: la única dirección alcanzable es la que el propio cliente descubrió
   de sí mismo. Y esa dirección cambia cada vez que la entrada expira y se
   renueva (NAT REBINDING), que es lo que rompe el TCP.
```

Solución	Costo
Port forwarding manual	El usuario tiene que abrirlo, y solo sirve para su IP, que puede cambiar.
Rendezvous / TURN	**Todo el tráfico pasa por el relay**: costo y latencia.
STUN + ICE + hole punching	No sirve con NAT simétrico ni con CGNAT que filtre lo no iniciado.
IPv6	Sin traducción; pero no todo el mundo lo tiene.

> **La frase del examen:** el P2P falla porque **detrás de un CGNAT el cliente no puede iniciar conexiones entrantes**: la tabla solo se llena cuando el cliente **sale**. Por eso todo P2P real usa un **servidor de rendezvous** (o TURN, que además relay) para que alguien publicable le diga a los demás cuál es su dirección alcanzable.

2.3 Ejercicio 3 — Por qué la NAT rompe IPsec en ESP

```
El ESP CIFRA LA CABECERA IP COMPLETA (direcciones de origen y destino) y el
receptor valida que coincida con la que ve por fuera.

El emisor mete IP_priv en su ESP; la NAT reescribe a IP_publica. El receptor
descifra, encuentra IP_priv y ve IP_publica por fuera.
⇐ LA AUTENTICACIÓN FALLA, y es IRREVERSIBLE: la NAT no puede recalcular el ESP.
```

Las dos soluciones, que son la respuesta completa: (1) **modo AH** (*Authentication Header*, protocolo 51), que autentica el datagrama IP entero pero **no lo cifra**, así que la NAT puede reescribir y recalcular; funciona, pero sin confidencialidad. (2) **NAT-T** (`RFC 3948` 🔍), que encapsula el paquete **ya traducido** dentro de un ESP con la dirección pública; funciona, pero agrega 24 bytes por paquete.

2.4 Ejercicio 4 — ¿Cuál de las tres soluciones corresponde a cada escenario?

Escenario	Solución
Servidor web interno alcanzable desde Internet	**1. *Port forwarding*** (puerto fijo e IP privada estable)
Máquina en la nube con **IP elástica** que se quiere exponer	**2. NAT sin traducción** (mapeo 1:1)
Teléfono VoIP detrás de la NAT que quiere recibir llamadas	**3. El servidor inicia** (el `REGISTER` de SIP)
WebRTC entre dos navegadores detrás de un CGNAT	**3 + TURN**
Dos equipos de la misma LAN que se quieren ver por la IP pública	**3 + *hairpinning***

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ ⭐ Examen 2011-02, Pregunta 4 (10 puntos) — NAT, capas y las tres soluciones

> **Fuente:** Examen `eirc1102` / `sirc1102` — 3 de febrero de 2011, Pregunta 4 (10 puntos). **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1102.pdf

Enunciado (texto literal del examen)

> **a)** Describa la técnica denominada NAT, explicando las modificaciones que un router NAT debe realizar sobre los paquetes salientes y entrantes.
>
> **b)** De una justificación para la siguiente frase: "*NAT no respeta el principio de aislación de capas*".
>
> **c)** Describa brevemente las tres soluciones planteadas en el teórico para atravesar un router NAT por parte de un cliente que se encuentra en una red con direcciones públicas y quiere contactar a un servidor que está en una red con direcciones privadas.

Sobre la solución oficial

El PDF de `sirc1102` **no trae la solución publicada de esta pregunta**: trae el enunciado y remite a las diapositivas del teórico. Las tres resoluciones de abajo son **solución propia**.

Parte a) — Qué es la NAT y qué cambia en cada sentido (≈4 puntos)

La **NAT** es una función de un **router de borde** que **reescribe las direcciones IP** de los paquetes, sustituyendo las direcciones **privadas** (que no se rutean en Internet) por una **dirección pública** del propio router. Para sostener **muchas sesiones simultáneas** con una sola dirección pública traduce además el **puerto** de origen: eso es la **PAT**.

```
⇐ SENTIDO SALIENTE  (LAN → Internet)

ETIQUETA IP                                TCP/UDP
origen      : IP PRIVADA   → IP PÚBLICA  ← SE CAMBIA
puerto origen: puerto PRIVADO → puerto PÚBLICO  ← SE CAMBIA
destino     : sin cambios                ← no se toca
puerto destino: sin cambios              ← no se toca

Además: se CREA una entrada en la tabla NAT
        (IP priv, port priv) → (IP pública, port público)

⇐ SENTIDO ENTRANTE  (Internet → LAN)

origen      : sin cambios                ← no se toca
puerto origen: sin cambios                ← no se toca
destino     : IP PÚBLICA   → IP PRIVADA   ← SE CAMBIA
puerto destino: puerto PÚBLICO → puerto PRIVADO  ← SE CAMBIA

El router busca la entrada por (IP pública, puerto destino). Si NO la
encuentra, DESCARTA el paquete.
```

El porqué de que el puerto destino sea la clave de búsqueda es lo que da el apartado b): todos los paquetes que entran tienen **la misma IP de destino** (la pública del router), y lo único que los distingue es el **puerto de destino**, campo de la cabecera TCP/UDP.

Para los 4 puntos: nombre de la técnica, por qué existe, tabla de campos por sentido, creación y consulta de la tabla, y el descarte.

Parte b) — Por qué la NAT no respeta el aislamiento de capas (≈3 puntos)

> El aislamiento de capas consiste en que **cada capa use a la de abajo como un servicio opaco**, y en que la capa 3 decida el reenvío mirando **solo la dirección IP destino**.
>
> Con NAT, un paquete que llega desde Internet tiene como destino la **dirección pública del propio router**, que es **única para toda la red interna**. Para saber a qué máquina de la LAN debe ir, el router tiene que consultar su tabla NAT, y esa tabla está indexada por el **puerto TCP/UDP de destino**.
>
> Por lo tanto el router, que es un dispositivo de **capa 3**, se ve obligado a **leer un campo de la cabecera de capa 4** para tomar su decisión de reenvío. Eso es una **invasión de capas**: el comportamiento de la capa 3 queda atado a un protocolo concreto de capa 4.
>
> Las consecuencias son: (i) **cualquier protocolo de capa superior que no sea TCP o UDP deja de funcionar** detrás de una NAT sin que el router lo sepa; (ii) se rompe la **extensibilidad**, porque al agregar una capa nueva hay que modificar también la capa 3; y (iii) se rompe la **transparencia extremo a extremo**, como se ve con IPsec en modo ESP, que cifra la cabecera IP y por lo tanto no tolera que se reescriban las direcciones.

Puntos clave: (1) la definición del aislamiento, (2) que el destino es la IP del router y hace falta el puerto, (3) que el puerto es de capa 4, (4) la consecuencia.

Parte c) — Las tres soluciones (≈3 puntos)

Las tres están desarrolladas en el **§1.5**, con diagramas y límites. En el examen:

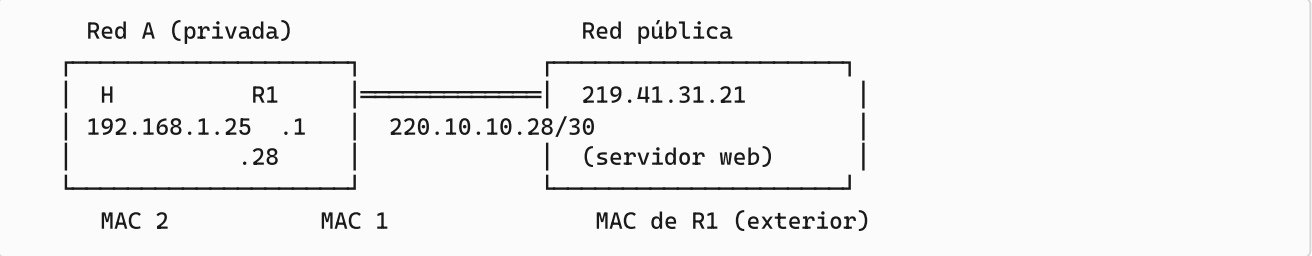
> **1) *Port forwarding* (NAT estática).** El administrador configura en el router un mapeo explícito: "todo lo que entre por el puerto público 8080, mandalo a `10.0.0.5:80`". La entrada de la tabla NAT es fija y siempre existe. Requiere IP privada estable en el servidor y conocer de antemano puerto y protocolo.
>
> **2) NAT sin traducción** (a veces *full-cone NAT* o *endpoint-independent mapping*). El router **no cambia nada** y deja pasar el paquete, con la IP de origen intacta. Sirve cuando el servidor tiene una dirección pública propia (por ejemplo una IP elástica en la nube) y lo que hace falta es que el router no reescriba. También requiere que el mapeo esté declarado de antemano.
>
> **3) El servidor inicia la conexión** (inversión de quién origina). El servidor privado **abre una conexión saliente**; el router guarda la entrada en su tabla NAT. A partir de ahí, cualquier paquete que **entre** desde Internet con ese mismo par `(IP pública, puerto público)` se traduce y se entrega al servidor. El cliente público "contesta" a una conexión que él no abrió. Es la base de SIP, ICE/WebRTC, STUN y TURN, y es la única que **no requiere configurar nada**.

3.2 ⭐ Examen 2011-03, Problema 2, apartado c — las modificaciones de R1

> **Fuente:** Examen `eirc1103` / `sirc1103` — 21 de febrero de 2011, Problema 2, apartado c. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1103.pdf

Contexto del problema (los apartados a y b están resueltos en la Clase 30)

> Una empresa tiene una red A (LAN Ethernet con direccionamiento privado) con 15 servidores y 70 puestos de trabajo, y se desea que puedan acceder a un servidor web en Internet con dirección IP `219.41.31.21`. Para ello el router `R1` implementa NAT utilizando su dirección IP pública.



La tabla de enrutamiento (resuelta en la Clase 30 §3): R1 tiene `192.168.1.0/24` y `220.10.10.28/30` conectadas, y `0.0.0.0/0` por `220.10.10.29`. El host H tiene `192.168.1.0/24` conectada y `0.0.0.0/0` por `192.168.1.1`.

Enunciado del apartado c (texto literal)

> **c)** Especifique las modificaciones que realizará R1 en los paquetes de datos que viajan entre Host H y Servidor Web. Considerar ambos sentidos.

Solución oficial (textual)

> **Sentido host H → Servidor Web:** "*Datagramas con dirección IP de origen 192.168.1.25 y dirección IP de destino 219.41.31.21. R1 sustituye la IP de origen por 220.10.10.30, y el puerto de origen (port #) por un puerto nuevo (new port #) asignado por R1.*" Y: "*El router R1 tendrá una tabla de NAT con los siguientes datos: (Dirección IP origen, port #), (220.10.10.30, new port #)*".
>
> **Sentido servidor Web → host H:** "*Datagramas con dirección IP de origen 219.41.31.21, dirección IP de destino 220.10.10.30, y puerto de destino new port #. A partir de la tabla de NAT, R1 sustituye la IP de destino por 192.168.1.25, y el puerto de destino por port #.*"

Análisis de la solución oficial

Lo que está bien: (1) nombra **exactamente los dos campos** que se cambian en cada sentido; (2) dice explícitamente que **se crea una tabla NAT** con la correspondencia de puertos, sin la cual la entrada no se puede resolver; (3) en el sentido entrante nombra los **tres campos que hay que mirar** antes de sustituir dos.

Lo que falta y son los puntos que se ganan: 1. **No da valores concretos** para el puerto asignado: dice "new port #" en vez de, por ejemplo, 50001. El enunciado no lo fuerza, pero **inventar un número concreto** muestra que entendiste que es una asignación del router. 2. **No dice qué pasa si no hay entrada en la tabla** (sentido entrante): el paquete **se descarta**. 3. **No dice qué NO se modifica.** Decir "la IP de destino en la salida y la IP de origen en la entrada **no se tocan**" es lo que hace la respuesta completa. 4. **No menciona el *timeout*** de la entrada, ni que se rehacen las tramas Ethernet.

Mi resolución completa del apartado c

Sentido 1 — H → Servidor Web (salida).

```
LLEGA A R1:  IP  origen 192.168.1.25  destino 219.41.31.21
             TCP origen 1024          destino 80
             Eth origen MAC 2         destino MAC 1 (R1, red A)

ACCIÓN: R1 busca (192.168.1.25, 1024) en la tabla NAT.
  · Si NO está → reserva un puerto público libre (ej. 50001), CREA la
    entrada y traduce.   · Si ya está → la reutiliza.

SALE DE R1:   IP  origen 220.10.10.30  destino 219.41.31.21   ← CAMBIA
             TCP origen 50001          destino 80            ← CAMBIA
             Eth origen MAC 1 → destino MAC de R1 en 220.10.10.28/30

ENTRADA CREADA: (192.168.1.25, 1024) → (220.10.10.30, 50001)  con TIMEOUT
⚠️ NO SE TOCA: la IP de DESTINO (219.41.31.21) ni el puerto de DESTINO (80).
```

Sentido 2 — Servidor Web → H (entrada).

```
LLEGA A R1:  IP  origen 219.41.31.21  destino 220.10.10.30
             TCP origen 80            destino 50001
             Eth destino = MAC de R1 en 220.10.10.28/30

BÚSQUEDA: (220.10.10.30, 50001) en la tabla NAT.
  ⚠️ por eso hace falta traducir PUERTOS: la IP de destino es la misma
     para todas las entradas.
  · Si NO está → R1 DESCARTA el paquete.
  · Si está → refresca el TIMEOUT y traduce.

SALE DE R1:   IP  origen 219.41.31.21  destino 192.168.1.25    ← CAMBIA
             TCP origen 80            destino 1024           ← CAMBIA
             Eth origen MAC 1 → destino MAC 2 (host H)

⚠️ NO SE TOCA: la IP de ORIGEN ni el puerto de ORIGEN. El servidor web no
   tiene por qué enterarse de que hubo NAT.
```

La tabla resumen, que es lo que el corrector busca:

Campo	Salida (H → S)	Entrada (S → H)
IP **origen**	`192.168.1.25` → **`220.10.10.30`**	`219.41.31.21` → sin cambio
puerto origen	`1024` → **`50001`**	`80` → sin cambio
IP **destino**	`219.41.31.21` → sin cambio	`220.10.10.30` → **`192.168.1.25`**
puerto destino	`80` → sin cambio	`50001` → **`1024`**
Tabla NAT	se **crea** la entrada	se **busca** y se refresca el timeout; si no está, **descarte**
Ethernet	MAC destino = MAC de R1 en `220.10.10.28/30`	MAC destino = MAC 2 (host H)

> **Nota sobre la oficial:** coincide exactamente con mi resolución en los dos sentidos. La oficial **no da valores concretos** para el puerto asignado y **no menciona el descarte**; esas dos son las adiciones que suman puntos.

3.3 ⭐ ⭐ Examen 2008-12, Problema 1 (30 puntos) — NAT para balanceo de carga

> **Fuente:** Examen `eirc0812` / `sirc0812` — 20 de diciembre de 2008, Problema 1 (30 puntos). **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0812.pdf

Enunciado (texto literal del examen)

> Se desea utilizar una variante del protocolo NAT para realizar balanceo de carga entre tres servidores instalados en una red privada. Los servidores tienen direcciones privadas `IP_1`, `IP_2`, `IP_3`, y publican un servicio en el puerto `PORT`. El servicio es conocido en Internet a través del sistema Acceso, router de la red privada, cuya dirección pública es `IP_SERVICIO`. Se utiliza el mismo puerto `PORT` para accederlo.
>
> Al llegar una solicitud de servicio a Acceso, se debe redireccionar la misma al servidor que se encuentre atendiendo menor número de requerimientos. Una vez establecida una correspondencia entre un cliente y un servidor, debe mantenerse mientras se mantenga el flujo de tráfico. Se considera que el flujo ha terminado cuando transcurrieron `TIMEOUT` segundos sin tráfico.
>
> **a)** Indique qué campos deben modificarse en los paquetes que llegan a Acceso para lograr el redireccionamiento especificado. ¿Es necesario modificar los encabezados de capa 4 o solamente los de capa 3? Justifique su respuesta.
>
> **b)** En base a la respuesta anterior, especifique las estructuras que necesita almacenar Acceso para realizar la traducción de direcciones y el balanceo de carga.
>
> **c)** Especifique, en un lenguaje de alto nivel, el procedimiento que debe ejecutar Acceso cada vez que recibe un paquete, para cumplir la funcionalidad descrita.

Solución oficial (completa, textual)

a)

> *"Solamente necesitamos modificar campos de capa 3. Cuando llega un paquete IP de un host externo, elegimos el servidor menos cargado y pisamos la IP de destino del paquete poniendo la IP del servidor elegido. Cuando un paquete viene de un servidor interno y va hacia fuera sólo es necesario pisar la IP de origen y poner la del router. No es necesario modificar los puertos de origen y destino de los paquetes, pero sí es necesario guardarlos en la tabla de NAT para poder enviar los datos de una misma conexión TCP siempre hacia el mismo servidor."*

b)

```
struct NATRegister {
  int32 srcIP;
  int32 dstIP;
  int16 srcPort;
  int16 dstPort;
  long  timestamp;
}
Map<{int32, int16}, NATRegister> NATTable;
// usamos una tabla indizada por el par {ip, puerto} de origen
Map<int32, int> cantidadConexiones;
// esta tabla nos indica cuántas conexiones tenemos por cada servidor
```

c)

```
void acceso() {
  NATTable = new Map<{int32, int16}, NATRegister>();
  cantidadConexiones = new Map<int32, int>();
  while (true) {
    PacketIP p = receiveFromNetwork();
    cleanTimeoutRegisters();
    if (p.srcIP in [IP_1, IP_2, IP_3]) {         // respuesta de un servidor
      NATRegister reg = NATTable.get(p.dstIP, p.dstPort);
      if (reg != null) reg.timestamp = now();
      p.srcIP = IP_SERVICIO;                      // ←— pisa el ORIGEN
      sendToNetwork(p);
    } else {                                       // viene de afuera
      NATRegister reg = NATTable.get(p.srcIP, p.srcPort);
      if (reg == null) {
        int32 srvIP = cantidadConexiones.findMinimum();
        reg = new NATRegister();
        reg.srcIP = p.srcIP;  reg.srcPort = p.srcPort;
        reg.dstIP = srvIP;    reg.dstPort = PORT;
        NATTable.put({p.srcIP, p.srcPort}, reg);
      }
      reg.timestamp = now();
      p.dstIP = reg.srcIP;                        // ←— BUG: debe ser reg.dstIP
      sendToNetwork(p);
    }
  }
}
void cleanTimeoutRegisters() {
  long maxTime = now() + TIMEOUT;
  for each NATRegister reg in NATTable {
    if (maxTime > reg.timestamp) {                // ←— BUG: condición invertida
      NATTable.remove(reg);
      cantidadConexiones[reg.dstIP]--;             // ←— pero NUNCA se suma
    }
  }
}
```

Análisis de la solución oficial

Lo que está muy bien: 1. **Responde el apartado a) con el criterio correcto** ("solamente campos de capa 3") y da la **razón profunda**: los puertos **no se modifican** pero **sí se guardan** en la tabla, "para poder enviar los datos de una misma conexión TCP siempre hacia el mismo servidor". **Esa frase vale los puntos del apartado.** 2. **La estructura del `NATRegister` es la correcta**: IP y puerto de **origen** (el cliente externo),

IP de destino (el servidor elegido), ambos puertos y un `timestamp`. 3. **El índice de la tabla es el correcto**: el par `{ip, puerto}` de **origen**, porque es lo único que llega sin traducir. 4. **La segunda estructura es la clave del balanceo**: un mapa servidor → cantidad de conexiones, con una operación de mínimo.

🔴 **Errores reales en el pseudocódigo de la oficial (señalarlos suma puntos):**

1. `p.dstIP = reg.srcIP;` **está al revés.** En la rama de "viene de afuera" hay que pisar **el servidor elegido**, o sea `reg.dstIP`. Con `reg.srcIP` el paquete vuelve al cliente externo con su propia IP de destino. La oficial **se contradice con su propio apartado a)**.
2. **Nunca se incrementa** `cantidadConexiones`**.** La tabla de carga se crea pero el código **no la suma** en ningún lado: el balanceo nunca elige el menos cargado.
3. **La condición del timeout está invertida.** `maxTime > reg.timestamp` compara la hora actual **más** el timeout contra el timestamp: si `timestamp` es mayor, la entrada es **nueva**. Lo correcto es `now() - reg.timestamp > TIMEOUT`.
4. `cleanTimeoutRegisters()` **se llama una vez por paquete**, es decir **O(n) por paquete**. Con miles de flujos activos corresponde barrer la tabla **periódicamente** o usar una cola ordenada por timestamp.
5. `NATTable.get(p.dstIP, p.dstPort)` le pasa dos argumentos a una tabla indexada por un **par**, y si un paquete de un servidor llega **sin entrada** (porque expiró a mitad de un flujo) el código lo reenvía igual con `srcIP = IP_SERVICIO` sin avisar.

Mi resolución completa (para los 30 puntos)

Parte a) — Qué campos se modifican

```
⇐ ENTRANTE (cliente externo → Acceso → servidor)

LLEGA:  IP origen = cliente externo ; IP destino = IP_SERVICIO
        TCP origen = cPort ;         TCP destino = PORT
        Eth destino = MAC de Acceso

ACCIÓN: si (IP origen, cPort) NO está en la tabla:
            srvIP = el servidor con MENOR cantidadConexiones
            crear reg = { src=(IP origen, cPort), dst=(srvIP, PORT), t=now() }
            cantidadConexiones[srvIP]++        ⟵ ¡FALTA EN LA OFICIAL!
        si ya está: se reutiliza la entrada, SIN volver a elegir servidor.

SALE:   IP origen = cliente externo   (NO se toca)
        IP destino = srvIP           ⟵ SE PISA
        TCP origen = cPort ; destino = PORT   (NO se tocan)
        Eth destino = MAC de srvIP

⚠️ POR QUÉ NO HAY QUE TOCAR LOS PUERTOS:
   El cliente pidió el puerto PÚBLICO (PORT). Si Acceso cambiase el puerto de destino, el
   cliente no reconocería la respuesta (su socket espera en PORT). ⇐ El TRUCO: se guardan
   los puertos REALES en la tabla y se usan para mandar la respuesta al servidor correcto.
   Como el puerto público coincide con el privado (PORT = PORT), el paquete sale sin
   traducción de capa 4. ⇐ OJO: esto solo funciona porque el enunciado dice "el mismo
   puerto PORT". Si el servidor escuchara en otro, habría que traducir capa 4.

⇐ ENTRANTE (servidor → Acceso → cliente externo)

    IP origen = srvIP                ⟵ SE PISA por IP_SERVICIO
    IP destino = cliente externo     (NO se toca)
    TCP origen = PORT ; destino = cPort   (NO se tocan)
```

La respuesta al "¿es necesario modificar los encabezados de capa 4?": No, solamente capa 3, porque los puertos se **guardan** en la tabla NAT y la correspondencia cliente-servidor se mantiene por sesión sin traducirlos. Es el mismo criterio que usa un balanceador que **no** quiere terminar TLS: termina solo en capa 3 o 4.

Parte b) — Estructuras de datos

```
1) REGISTRO DE UNA SESIÓN
     struct NATRegister {
         ip_addr srcIP;     // IP del cliente externo (no se traduce)
         ip_addr dstIP;     // IP del servidor PRIVADO elegido
         uint16  srcPort;   // puerto origen real del cliente
         uint16  dstPort;   // puerto destino real (PORT)
         long    timestamp; // último instante con tráfico
     };

2) TABLA DE SESIONES, indexada por el par {srcIP, srcPort}
     Map<{ip_addr, uint16}, NATRegister>  NAT;

3) CONTADOR POR SERVIDOR (lo que permite el balanceo)
     Map<ip_addr, int>  Carga;   // IP_1 → 120, IP_2 → 87, ...

4) (recomendable) COLA DE EXPIRACIÓN ordenada por timestamp, para no barrer
   toda la tabla en cada paquete.
```

Por qué la clave es `{srcIP, srcPort}` y no `dstIP`: porque `dstIP` es siempre `IP_SERVICIO`, la misma para todas las entradas, y no distingue nada. Lo único que **llega intacto** y es único por cliente es `(IP origen, puerto origen)`. Si un cliente abre dos conexiones simultáneas tiene dos puertos de origen distintos, y por eso van a entradas (y a servidores) distintos.

Por qué hace falta `dstPort` en el registro si no se traduce: para distinguir tres servidores que **escuchan en el mismo puerto**, que es justo el caso del enunciado.

Parte c) — El procedimiento

```
// ---- Estado persistente ----
Tabla<Par{ip,uint16}, NATRegister>  NAT;      // clave = (srcIP, srcPort)
Tabla<ip, int>                      Carga;     // servidor → conexiones activas
Cola<timestamp, Par>               Expiran;   // barrido eficiente

bool esServidorPropio(ip a):
    return (a == IP_1 || a == IP_2 || a == IP_3)

ip elegirServidor():
    mejor = IP_1
    for srv in {IP_1, IP_2, IP_3}:
        if Carga[srv] < Carga[mejor]:  mejor = srv      // desempate por IP
    return mejor

void refresh(NATRegister r):
    r.timestamp = now()

void limpiarExpirados():       // barra solo lo vencido, con una cola ordenada
    while Expiran no vacía and Expiran.menor().timestamp < now() - TIMEOUT:
        k = Expiran.extraer()
        r = NAT.buscar(k)
        if r != null and r.timestamp < now() - TIMEOUT:
            NAT.eliminar(k)
            Carga[r.dstIP] = Carga[r.dstIP] - 1      ←── ¡FALTA EN LA OFICIAL!

void acceso():
    NAT = nueva Tabla()
    Carga = { IP_1→0, IP_2→0, IP_3→0 }
    Expiran = nueva Cola()

    while (true):
        p = recibirPaqueteDeLaRed()
        limpiarExpirados()

        if esServidorPropio(p.srcIP):
            // ===== SENTIDO SALIENTE (servidor → cliente) =====
            r = NAT.buscar({ p.dstIP, p.dstPort })     // dstIP = IP_SERVICIO
            if r == null:
                descartar(p)      // el cliente cerró y la entrada expiró
                continue
            refresh(r)            // la sesión sigue viva
            p.srcIP = IP_SERVICIO                      // ←── SE PISA el origen
            enviarPaqueteALaRed(p)                     // destino y puertos, igual

        else:
            // ===== SENTIDO ENTRANTE (cliente → servidor) =====
            k = { p.srcIP, p.srcPort }
            r = NAT.buscar(k)
            if r == null:
                // ===== flujo NUEVO: acá está el BALANCEO =====
                srv = elegirServidor()
                r = new NATRegister()
                r.srcIP = p.srcIP ; r.srcPort = p.srcPort
                r.dstIP = srv                          // ←── EL SERVIDOR ELEGIDO
                r.dstPort = p.dstPort
                NAT.insertar(k, r)
                Carga[srv] = Carga[srv] + 1            ←── ¡FALTA EN LA OFICIAL!
            refresh(r)
```

```
            p.dstIP = r.dstIP                          // ◄── SE PISA el destino
            enviarPaqueteALaRed(p)
```

Puntos (30): (a) qué campos y por qué no los puertos (8), (b) las dos estructuras y la justificación de la clave (10), (c) el procedimiento con los dos sentidos, el *timeout* y el contador (12).

> **Nota final sobre la oficial:** la solución oficial tiene **cuatro errores reales** en el pseudocódigo (el `p.dstIP = reg.srcIP` invertido, el contador que nunca se incrementa, la condición de *timeout* invertida y el barrido O(n) por paquete). **El enunciado, la parte a) y la parte b) de la oficial están bien.** Si tu respuesta los señala, sumás puntos: a la cátedra le importa que entiendas el mecanismo, no que copies.

PARTE 4 — CIERRE

📌 Resumen

1. **NAT** = traducción de direcciones; existe por la **escasez de IPv4** y porque las **direcciones privadas no se rutean**. No es seguridad, es escasez.
2. La traducción de **dirección sola** soporta **una** sesión por IP pública; la **PAT/NAPT** soporta miles. En FING, "NAT" significa **PAT**.
3. **Tres fases: salida** (se crea la entrada), **entrada** (se busca por `(IP pública, puerto destino)`; si no está, **se descarta**) y **timeout**.
4. **Salida:** cambian **IP origen** y **puerto origen**. **Entrada:** cambian **IP destino** y **puerto destino**. Nada más.
5. **Rompe el aislamiento de capas** porque un router de capa 3 tiene que **leer el puerto TCP/UDP** (capa 4) para saber a qué host interno va.
6. **Las tres soluciones: port forwarding**, **NAT sin traducción** (1:1 declarado) y **el servidor inicia** (la entrada se crea sola; base de SIP, ICE y TURN).
7. **Hairpinning** permite que un cliente de la LAN alcance a otro por la IP pública del router.
8. **CGNAT**: sus tres problemas son **conectividad entrante rota**, **señalización complicada** e **IPsec ESP roto**.
9. Con IPv6 **no hay traducción**: por eso la meta es **eliminarla**, y el `Connection ID` de QUIC hace que el *NAT rebinding* ya no rompa las conexiones.
10. **IPv4 se agotó**: el RIPE hizo su última asignación `/22` el **25-nov-2019**, y el acceso a Google por IPv6 superó el **50 % el 28-mar-2026**.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. En la tabla NAT, dos sesiones salen del mismo host `192.168.1.25` con puertos de origen 1024 y 1025. ¿Cuántas entradas hay y por qué dos?

<details>
<summary>Respuesta</summary>
</details>

2. En el sentido de entrada, ¿qué pasa con un paquete cuyo par `(IP pública, puerto público)` **no está** en la tabla?

<details>
<summary>Respuesta</summary>
</details>

3. ¿Por qué la NAT rompe el aislamiento de capas? Formulá la respuesta en dos frases.

<details>
<summary>Respuesta</summary>
</details>

4. ¿Cuál es el problema del P2P detrás de un CGNAT, y por qué el *hole punching* no siempre lo resuelve?

▶ Respuesta

5. ¿Qué tres cosas hay que mantener en un balanceador de carga tipo `2008-12 Problema 1`? Mencioná los dos errores del pseudocódigo oficial.

▶ Respuesta

6. Un amigo te dice: "*configurá port forwarding y el problema de que no te puedan llegar adentro desaparece para siempre*". ¿Qué le responderías?

▶ Respuesta

7. ¿Qué cambia con IPv6, y por qué la meta del IPv6 es eliminar la NAT?

▶ Respuesta

8. ¿Qué es el *hairpinning* y por qué hace falta?

▶ Respuesta

🔗 Referencias

Exámenes - `eirc1102` / `sirc1102` (3-feb-2011) — **P4 (10 pts): NAT, por qué rompe el aislamiento de capas, y las tres soluciones**. §3.1. - `eirc1103` / `sirc1103` (21-feb-2011) — **Problema 2 c: qué campos cambia R1 en cada sentido + tabla NAT**. §3.2. (Los apartados a y b están en la Clase 30.) - `eirc0812` / `sirc0812` (20-dic-2008) — **Problema 1 (30 pts): variante de NAT para balanceo de carga entre tres servidores privados**, con pseudocódigo. §3.3. - `sirc1003` (mar-2010) P1: subredes privadas y NAT. `sirc0903` (feb-2009) Problema 2: NAT como proxy.

Bibliografía - Tanenbaum & Wetherall, *Redes de computadoras*, 5ª ed., cap. 5.

RFCs verificados en `ACTUALIZACIONES.md` - `RFC 8200` — IPv6 (2017, *Internet Standard*): por qué no hace falta traducción. - `RFC 6146` — NAT64 (2011) y `RFC 6147` — DNS64 (2011), con `RFC 6052` y `RFC 8880`. - `RFC 7915` — SIIT (2016), que **obsoleta** a la `RFC 6145`. - `RFC 6333` — DS-Lite (2011): dual-stack *stateless*. - `RFC 9000` — QUIC (2021) §5.1: el **Connection ID** hace que el *NAT rebinding* no rompa las conexiones. - `draft-ietf-v6ops-464xlat` — 464XLAT, **sigue en borrador**.

RFCs NO verificados (no los cites sin confirmar) - NAT tradicional (`RFC 3022`), espacio compartido del CGNAT (`RFC 6598`), requisitos del CGN (`RFC 6888`), NAT-T (`RFC 3948`) y detección de *NAT rebinding* (`RFC 8443`): **verificar**.

Clases relacionadas - Clase 26, 27 y 29 (IPv4 y subnetting) · Clase 30 (enrutamiento y ARP) · Clase 36 (IPv6) · Clase 44 (VoIP).